POLAR-NAV AI

AI-Enabled Antarctic Sea-Ice, Iceberg Trajectory and Navigation Decision Support System

Ministry of Earth Sciences · National Centre for Polar and Ocean Research (NCPOR)
Prototype report, version 1.0.0 · generated 07 September 2026, 11:13 UTC

13,581	6,611	108	25+
LINES OF PYTHON	LINES OF TYPESCRIPT	AUTOMATED TESTS	API ENDPOINTS

1. The problem statement, as issued

Problem Statement ID	26059
Title	AI-Enabled Antarctic Sea-Ice, Iceberg Trajectory, and Navigation Decision Support System
Organisation	Ministry of Earth Sciences (MoES)
Department	National Centre for Polar and Ocean Research (NCPOR)
Category and theme	Software · Transportation & Logistics
Event	Smart India Hackathon 2026

> **The ask, as written.** Develop an AI/ML-enabled decision support platform capable of forecasting Antarctic sea-ice concentration, predicting iceberg trajectories, and identifying safe and fuel-efficient navigation routes for research vessels using satellite, oceanographic and meteorological datasets.

The ten capabilities the statement requires, and where each one lives

#	Required capability	Status	Implemented in
1	Environmental conditions: ice concentration, thickness, drift, wind, currents, temperature, sea state	Built	`core/environment.py`, `core/sea_ice.py`
2	Sea-ice forecasting, with drift, divergence and compression, verified against persistence	Built	`core/sea_ice.py`
3	Iceberg trajectory prediction with ensembles and closest point of approach	Built	`core/iceberg_tracker.py`
4	Ship performance in ice: resistance, attainable speed, power, fuel, CO2	Built	`core/lindqvist_model.py`
5	POLARIS safety assessment with ice-class risk values, and the arithmetic shown	Built	`core/polaris_risk.py`
6	Route optimisation against an ice-blind baseline, respecting POLARIS and iceberg hazards	Built	`core/route_optimizer.py`
7	Near-field radar growler detection, ice against sea clutter, with confidence and alerts	Built	`core/growler_radar.py`
8	Voyage monitoring, changing conditions, alerting and re-routing	Built	`core/voyage.py`
9	Offline operation, with no cloud service, external API or online map tiles	Built	whole stack; bespoke map engine
10	ECDIS interoperability: GPX, GeoJSON and an S-411-shaped ice overlay	Built	`services/exporters.py`

All ten are implemented and exercised by the automated test suite and the diagnostic sweep. Nothing in the required scope is stubbed.

2. What existed before, and what was built

A first-pass prototype existed: roughly 700 lines of Python across five modules, a command-line demonstration and ten tests. It had the right shape and the wrong internals. The work described in this report was an audit and a rebuild, and the audit found defects that mattered more than the missing features did.

Area	What was there	What it is now
The headline number	The saving was computed as `baseline = optimised × 1.22`. A constant. It reported the same 22 percent for any ship, any route and any ice, including a route that saved nothing.	Two independent routes are planned and **both sailed through identical physics**. The saving is the difference between them, and it is allowed to come out negative.
Land	No land mask at all. Nothing prevented a waypoint being placed on the Antarctic continent.	97 real Natural Earth coastline polygons as a hard constraint, answering 25,000 queries a second.
Ship speed	Assumed by the planner.	Solved from the Lindqvist power and propeller-thrust balance on every route segment.
POLARIS table	Approximate risk values and non-standard ice-type boundaries.	The official MSC.1/Circ.1519 table transcribed in full: 12 ice classes by 11 WMO ice stages.
Lindqvist model	Four transcription errors, including a bending term that was dimensionally wrong and backwards in Young modulus, making stiffer ice easier to break.	Corrected and validated against published results, with open-water resistance added so the curve stays continuous as ice thickness goes to zero.
Iceberg drift	A single constant velocity extrapolated in a straight line for the whole forecast.	RK4 integration of the full momentum balance with deterioration, perturbed ensembles and uncertainty radii.
Destinations	Routed to station coordinates, including Maitri, which is 80 km inland and unreachable by ship.	Every destination carries a seaward anchorage, validated against the real coastline at start-up.
Interface	None.	A React bridge console with a bespoke EPSG:3031 canvas map engine and six screens.
Machine learning	None, although the presentation claimed three AI engines.	Three models trained by one command, with honest results: one works and is used, two lose to the physics and are not.
Verification	10 tests, one of which asserted the fabricated saving.	108 tests, an 86-check diagnostic sweep, and an automated browser drive of the real interface.

3. How it helps, and who it helps

Who	What changes for them
The ice navigator on watch	Replaces a chart that is twelve to twenty-four hours old with a risk surface computed for *this* hull. The speed shown is what the ship can actually make in the ice ahead, not an estimate. When conditions turn, the alert names the hazard and the action rather than leaving the officer to infer it from numbers.
The master deciding whether to enter the pack	A defensible answer to whether the entry is permitted. POLARIS is enforced as a hard constraint, so a route through prohibited ice is never offered, and the arithmetic behind the risk score is shown so the decision can be justified afterwards.
The expedition leader	Six to seventeen days recovered inside a ninety-day window, measured across three legs. In a season that short, days are science, and a missed relief window means a station waits another year.
NCPOR planning the season	The cost of the current fleet becomes measurable rather than arguable. Plan the same passage with ORV Sagar Nidhi, the chartered Golovnin and a notional polar research vessel, and the planner shows precisely where each has to stop.
The ship operator	Charter time dominates the cost of a polar resupply, so transit days saved are the dominant economic benefit. Fuel and CO2 are reported per voyage, and the fuel figure stays honest even when it is unfavourable.
India, strategically	An open, auditable, offline-capable stack with no foreign commercial dependency, supporting obligations under the Antarctic Act 2022 and removing the need for commercial weather-routing subscriptions.

The mechanism, in one sentence. Ice makes a ship slow, and a slow ship in converging ice is a ship that gets stuck. The system finds the track where the ice is thin enough that the hull keeps moving, proves that track is permitted under the Polar Code, and keeps checking as conditions change.

Where it does not help, stated plainly. It will not save fuel on every route: on two of the three legs measured, the safe route burns more, because going around ice costs distance. It does not replace the ECDIS, the ice navigator, or an official ice chart. And it runs on simulated environmental fields, so it demonstrates a method rather than delivering an operational forecast.

4. What this is, in one page

Every austral summer India sends the Indian Scientific Expedition to Antarctica to resupply its two stations, **Maitri** and **Bharati**. The sailing window is about ninety days. Sea ice decides whether the ships make it, and the ice charts available today are largely manual and twelve to twenty-four hours old. Getting it wrong means burning fuel ramming ice, missing a relief window, or being beset — trapped as the ice closes in.

POLAR-NAV AI plans a route through that ice which is safe under the IMO Polar Code, and proves the benefit by measuring it. The central design decision is this: **the system plans two routes, not one**. It plans the route a ship would sail with no ice information at all, and the route it recommends. It then sails *both* through identical physics. The difference between them is the benefit, and because it is a measurement rather than an assumption, it is allowed to come out unfavourable — and sometimes does.

> **Why that matters.** The prototype this replaced computed its headline figure as `baseline = optimised × 1.22` and reported "22% fuel saved". That is a constant dressed as a result: it would report the same saving for any ship, any route and any ice, including a route that saved nothing at all. Removing it is the single most important change in this build.

5. What is real and what is simulated

This is the question every reviewer should ask first, so it is answered before anything else.

Layer	Status	Source	Note
Coastline	real	Natural Earth 1:50m physical land (public domain)	97 polygons, 1698 vertices, used as the hard land mask.
Stations	real	NCPOR published station coordinates	Maitri and Bharati positions are the published values.
Iceberg Catalogue	real-seed	US National Ice Center Antarctic iceberg tracking database	Real berg names and calving origins; positions are seeded then propagated by the drift model.
Polaris Matrix	real	IMO MSC.1/Circ.1519 Risk Value tables	Transcribed risk values; not simulated.
Sea Ice Field	synthetic	stands in for OSI-SAF OSI-401-b / AMSR2 / Sentinel-1 SAR	Physically-shaped seeded field. The models consuming it are real; the field is simulated.
Atmosphere Ocean	synthetic	stands in for ECMWF ERA5/HRES and CMEMS GLOBAL_ANALYSISFORECAST_PHY_001_024	Seeded synoptic + climatological forcing. Simulated.

> **The models are real; the weather is simulated.** The ship physics, the POLARIS risk tables, the Antarctic coastline, the geodesy and the iceberg drift equations are genuine. The sea-ice, wind, current and wave fields are physically-shaped simulations standing in for OSI-SAF, AMSR2, Sentinel-1, ERA5 and Copernicus Marine products. Every API response carrying a simulated field sets `is_synthetic: true` and names the product it replaces. All reported skill is therefore *simulated-environment* skill, and must be described that way. Swapping in live data is a data-loader change, not a model change.

6. Measured results

Leg	Distance (nm)	Time (h)	Fuel (t)	Fuel saved	Time saved	Min. risk index
Cape Town to Bharati	2,853 → 3,018	719 → **306**	301 → 292	+2.84%	413 h	6 → **12**
Cape Town to Maitri	2,174 → 2,569	664 → **495**	239 → 258	-7.98%	169 h	5 → **10**
Hobart to Bharati	2,703 → 2,959	432 → **280**	302 → 309	-2.35%	151 h	9 → **12**

Each row is two independent optimisations sailed through identical physics. Planning time 4–22 s per leg on a laptop CPU.

> **The result we did not expect, and did not hide.** On two of the three legs the safe route burns *more* fuel, because going around the ice adds distance and the distance costs more than the ice avoided. Re-weighting the optimiser toward fuel barely moves it: on the Maitri leg the penalty stays near 8% whichever way the weights are set, because it is a property of the geography and the ice, not of the search.
>
> **So the honest headline for this system is time and safety, not fuel.** It saves six to seventeen days inside a ninety-day window and improves the worst-case safety margin on every single leg. Charter time dominates the cost of a polar resupply, so days are worth more than tonnes. The planner emits an explicit warning whenever the recommended route costs more fuel, stating that it is being recommended because it is safer, not cheaper.

7. Built for the Indian programme, specifically

This is not a generic polar router with Indian place names attached. The programme's real constraints are inside the model.

The two stations are supplied in completely different ways

- **Maitri** sits about 80 km inland on the Schirmacher Oasis. No ship can reach it. Cargo is landed on the shelf ice at India Bay and hauled inland by convoy.
- **Bharati** is coastal but sits behind the Prydz Bay fast ice, with the Amery Ice Shelf calving tabular bergs into the approach.

Every destination therefore carries a **validated seaward anchorage**, checked against the real coastline at start-up, so the planner can never be given a destination a ship cannot reach.

The season is a hard constraint

Austral summer, roughly December to March. The sailing window is about 90 days. Days saved inside it are days of science, and days lost can mean a station goes unrelieved for a year, so transit time is the metric that matters most to this programme - more than fuel. The environment model's reference date is **26 November (day 330), the southbound departure** — deliberately the departure, not the February ice minimum, because late November is when the pack is still extensive and the routing problem is hardest.

The fleet, including the gap in it

Vessel	Ice class	LOA (m)	Power (MW)	Open water	0.6 m ice	1.0 m ice	1.5 m ice
MV Vasiliy Golovnin (NCPOR charter)	PC5	167	13.5	16.8	10.3	6.2	2.6
Arc7 resupply vessel (generic)	Arc7	169	13.0	16.0	11.1	7.7	4.5
ORV Sagar Nidhi (NIOT / MoES)	PC7	104	5.6	14.7	7.5	3.3	beset
Indian Polar Research Vessel (planned, notional design)	PC4	129	12.0	16.2	11.1	7.5	4.2

Attainable speed in knots at 6/10 ice concentration, solved from the power and propeller-thrust balance.

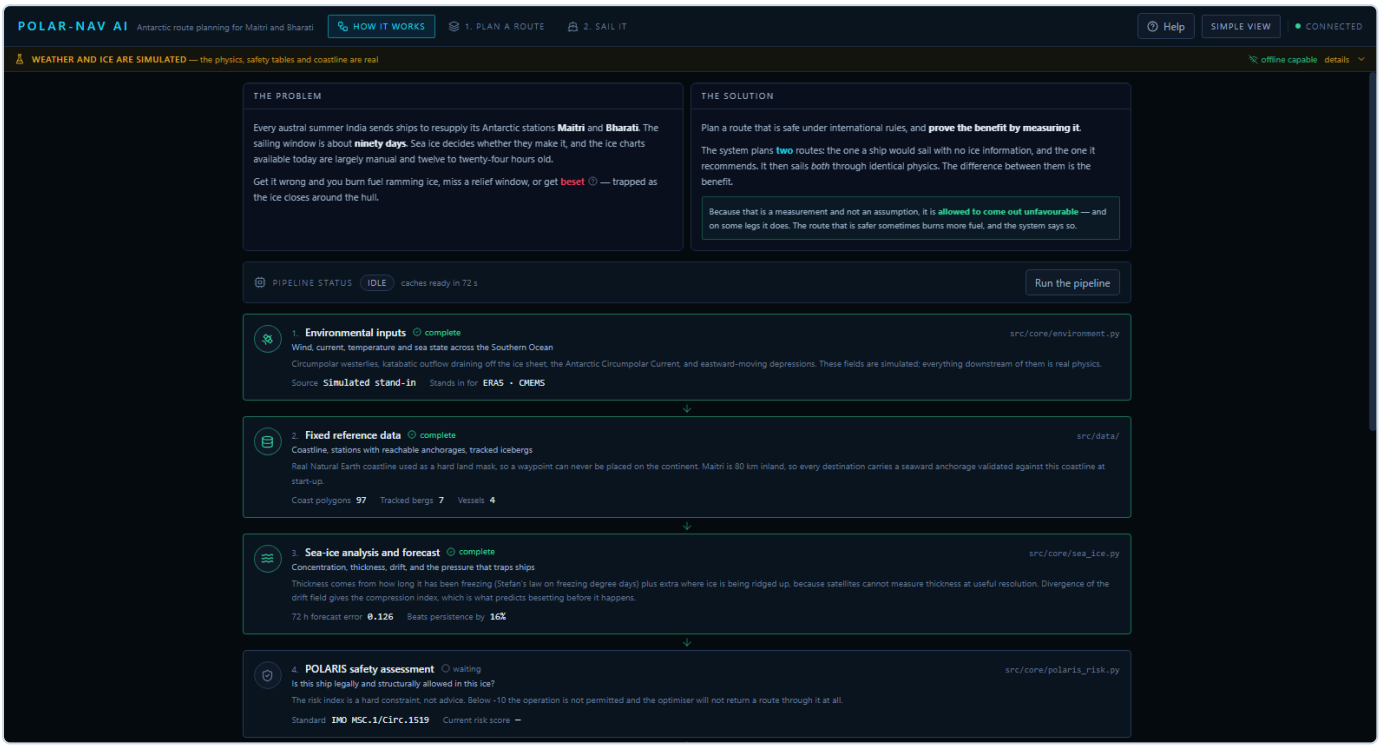
> **A quantified case for India's own polar vessel.** India has no dedicated polar research vessel. This system lets the consequence be measured rather than argued: plan the same passage with each hull and compare where the planner refuses to go. In 1.5 m ice at 6/10 concentration, Sagar Nidhi cannot make way at all, the chartered Golovnin manages about 2.6 knots, and a notional PC4 polar research vessel manages about 4.2. That difference is the business case, in physics.

The legal frame is enforced, not merely cited

- **Indian Antarctic Act, 2022** — Domestic law governing Indian activity in Antarctica. Requires a permit for vessel operations and imposes environmental obligations that a route planner has to respect, not merely note.
- **Antarctic Treaty and the Protocol on Environmental Protection** — Antarctic Specially Protected Areas must be avoided or entered only under permit; minimising time spent operating in them is a planning objective.
- **IMO Polar Code (MSC.385(94)) and POLARIS (MSC.1/Circ.1519)** — Sets the operational limits this system enforces as a hard constraint: a route may not be recommended through ice where POLARIS prohibits operation.
- **IMO Polar Code heavy fuel oil provisions** — Antarctic operations run on marine gas oil. The fuel and emissions model in this system is MGO throughout, at 3.206 kg CO_2 per kg fuel.

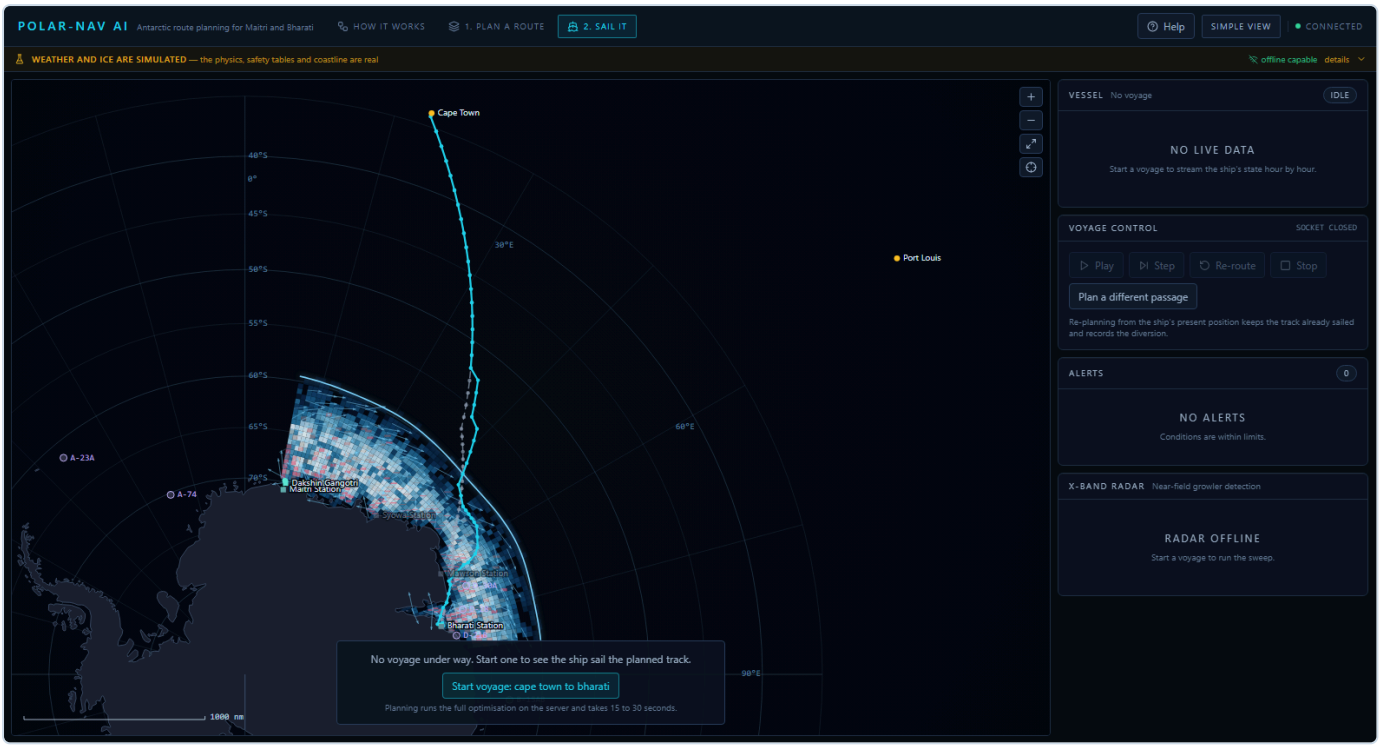
8. The prototype, as it actually runs

These are screenshots of the running application, captured by driving it through a real browser rather than mocked up. The interface has six screens; four are shown here.

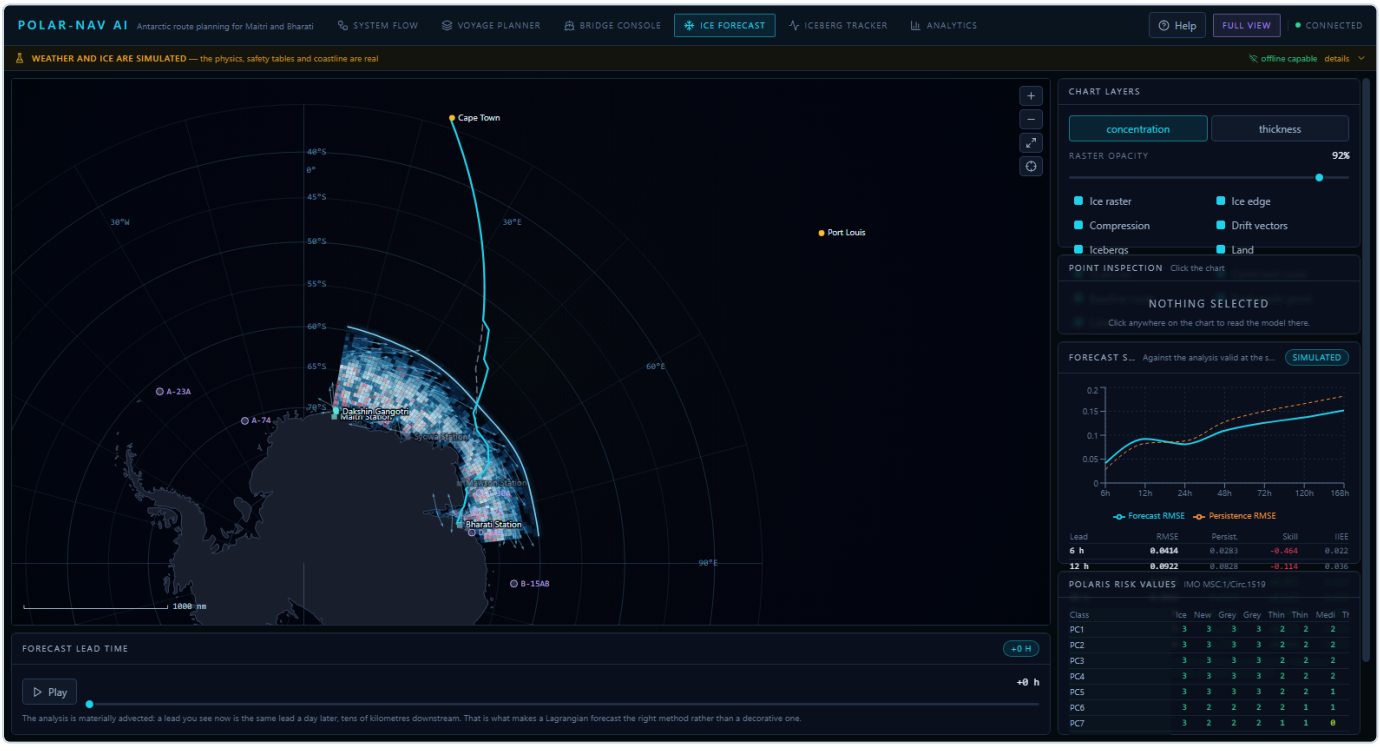


How it works. The landing screen states the problem and the solution, then shows the pipeline as eight stages with live numbers from the running backend. Stages light up as they compute.

Voyage planner. Cape Town to Bharati planned. Both routes are drawn on the chart, the solid track being the recommendation and the dashed one the route a ship would sail with no ice data. The verdict is stated in a sentence before any table.



Bridge console. The chart in EPSG:3031 Antarctic Polar Stereographic with the real coastline, the sea-ice raster, tracked icebergs and station labels. The rail carries the live vessel state, the alert feed and the radar scope.



Ice forecast. Lead time scrubbed from 0 to 168 hours with playback, layer toggles, point inspection, and the forecast skill measured against persistence.

> **Verified by driving it, not by assuming.** An automated Chrome DevTools session loads the application, dismisses the guide, walks every screen, presses Plan, waits for the real optimisation to finish and reads the result off the page. The run reports **zero console errors, zero uncaught exceptions and zero failed requests**, with the map canvas correctly sized at 1196 by 754 pixels.
>
> That check earned its place immediately: it found the map rendering blank on every screen. The canvas backing store was being sized one pixel tall, and nothing else in the project - not the type checker, not the tests, not any endpoint - could have caught it.

9. How it was built

Audit first, then rebuild

The work did not start by writing features. It started by reading the existing prototype and running it, which is how the fabricated saving was found: the fuel figure was traced backwards until it turned out to be a constant. Several other defects surfaced the same way, and they set the priority for everything that followed. Building new capability on top of a model that lies about its own results would only have made the lie harder to find.

The rule that shaped every decision

> **No number appears on screen unless something computed it at request time.** Every figure had to be traceable to a model that runs, and every model had to be checkable against something independent - a published result, a physical bound, or a baseline it has to beat. Where a claim could not survive that test, the claim was changed rather than the test.

Built in dependency order

Foundations first, because everything downstream samples them, and because a mistake in the geodesy would have quietly poisoned every result above it.

```
1.  Geodesy, land mask, constants       haversine, EPSG:3031 with a verified inverse,
                                        97 real coastline polygons behind a fast index
2.  Environment and sea ice             the fields everything else samples
3.  POLARIS and Lindqvist               pure, testable, checkable against published tables
4.  Icebergs and radar                  RK4 momentum balance; honest detection with misses
5.  Route optimiser                     the two-search design and the measured baseline
6.  Voyage engine, storage, exports     the passage as it unfolds, and ECDIS interoperability
7.  API                                 contract frozen here, so the interface could be built
8.  Bridge console                      map engine first, then the shell, then the screens
9.  Machine learning                    added last, on purpose: the physics had to stand alone
10. Documentation and verification      written from measurements, not from intentions
```

Parallel work against a frozen contract

Three self-contained pieces were developed in parallel by specialist agents working to a written specification: the Lindqvist resistance model with the radar simulation, the machine learning package, and the frontend. This only worked because the API contract was frozen and written down first, in `docs/PROTOTYPE_BUILD_SPEC.md`, before any of them started. Integration was done by hand, and it immediately earned its keep: the planner and the voyage engine had been given two different speed solvers, and only reconciling them by hand exposed that the planner was certifying routes that would beset the ship.

Four layers of verification, each catching what the others could not

Layer	What it is	What it caught that nothing else did
Unit and property tests	108 tests pinning physics bounds, monotonicity, projection round-trips and determinism	That the forecast must beat persistence, and that the reported saving must equal the difference between the two route evaluations
Diagnostic sweep	86 checks across every module, every endpoint including error paths, the full voyage lifecycle and determinism	Phantom sea ice at 30 S, created by noise applied where no ice existed
Type checking	TypeScript in strict mode against types derived from the running API	Schema drift: the backend had started returning fields the client did not know about
Driving the real interface	An automated Chrome DevTools session that loads the app, walks every screen, presses Plan and reads the result off the page	The map rendering blank on every screen, with the canvas sized one pixel tall

That last row is the important one. The build passed, the types checked, all 108 tests passed and every endpoint returned 200 - and the central feature of the application was invisible. No amount of testing the parts would have found it. Someone, or something, had to look at the screen.

Everything regenerates from one command

```
python -m pytest tests/ -q          108 tests
python -m src.cli                   the whole system, in a terminal, no browser
python -m scripts.train             retrain all three models, rewrite models/metrics.json
python -m scripts.make_report       regenerate this document from live model runs
.\start.ps1  /  ./start.sh          install, warm the caches, serve both halves
```

This report is itself generated: it re-plans all three legs, re-measures forecast skill and the bandwidth budget, and rebuilds the fleet table every time it is produced. A claim that stops being true cannot quietly survive in it.

What was corrected along the way, including our own claims

- The 15 to 22 percent fuel saving in the original blueprint **was not reproduced**, so the documentation now states the measured range and explains why time and safety are the honest headline instead.
- A preset vessel called *RV Himadri-class* was invented. Himadri is India's *Arctic* station, not a ship. It was replaced with ORV Sagar Nidhi, which is real.
- A season note claimed the models default to a mid-January reference date. They default to 26 November. The text was corrected rather than the model, because departure-season ice is the harder and more honest case to demonstrate.
- Two of three trained models were found to lose to the physics they were meant to improve. They are reported as failures and excluded from the serving path.

> Each of those was a claim this project had already made in writing. Finding them was the point of the audit discipline, and correcting them in public is what makes the remaining numbers worth trusting.

10. Architecture, top to bottom

```
SATELLITE AND REANALYSIS INPUTS          (simulated in this prototype)
Sentinel-1 SAR | AMSR2 | ERA5 | CMEMS | USNIC iceberg database
                         |
                         v
DATA LAYER               src/data/
Antarctic coastline as a hard land mask (97 polygons, Natural Earth 1:50m)
Stations with validated seaward anchorages | Iceberg catalogue | Programme context
                         |
                         v
PHYSICS CORE             src/core/          (pure functions, no I/O)
environment.py    westerlies, katabatic outflow, ACC, synoptic lows
sea_ice.py        concentration, Stefan thickness, drift, divergence, forecast
polaris_risk.py   IMO MSC.1/Circ.1519 risk index
lindqvist_model.py  ice resistance, powering, attainable speed
iceberg_tracker.py  RK4 momentum balance, melt, ensembles, closest approach
growler_radar.py  X-band PPI with honest misses and false alarms
route_optimizer.py  risk-constrained multi-objective A*, twice
voyage.py         hour-by-hour simulation, alerting, re-routing
                         |
                         v
MACHINE LEARNING         src/ml/            (optional; physics never depends on it)
Trained forecaster, drift residual corrector, growler classifier
                         |
                         v
SERVICES                 src/services/
SQLite voyage store | GPX / GeoJSON / S-411 export | bandwidth measurement
                         |
                         v
API                      src/api/           FastAPI, 25+ endpoints + WebSocket
                         |
                         v
BRIDGE CONSOLE           frontend/          React + custom EPSG:3031 canvas map
Plan | Sail | Ice forecast | Icebergs | Analytics
```

Dependency direction is strictly one-way. `core` depends on `data` and never the reverse; `services` and `api` depend on `core`; `ml` depends on `core` and **nothing depends on** `ml`, so the physics path runs unchanged if no model has ever been trained.

11. Technology stack, and why each choice

Choice	Why this rather than the obvious alternative
Python 3.11	The polar science ecosystem (xarray, NetCDF4, GDAL, Copernicus toolboxes) is Python. Ingesting real ERA5 and OSI-SAF products later means staying where those libraries live.
FastAPI + Pydantic v2	A typed schema and an OpenAPI document for free, so the frontend derives its types from the running server rather than guessing. Native WebSocket support for the live voyage stream.
NumPy / SciPy	Fields are evaluated on grids of thousands of points. Vectorisation took route planning from 24 s to 15 s and the iceberg ensemble from 109 s to 4 s.
scikit-learn, *not* PyTorch	The trained models are tabular and fit in seconds on a CPU. A bridge PC has no GPU. A deep network would add a heavy dependency and an ONNX export step for no measured gain.
SQLite	The shipboard console is one rugged PC with no infrastructure behind it. A file-backed database with write-ahead logging works there unchanged and at NCPOR headquarters too.
React 18 + Vite + TypeScript (strict)	Strict typing against generated API types means a backend field rename breaks the build rather than the demonstration.
Custom canvas map engine	The most important frontend decision. Mapbox, MapLibre and Leaflet all assume Web Mercator, which is unusable at 70°S, and all assume a reachable tile server, which does not exist at sea. The map projects to EPSG:3031 in TypeScript, mirroring the backend exactly, and draws our own model output.
Zustand + TanStack Query	Server state (slow, cacheable route plans) and client state (layer toggles, playback) are different problems and get different tools.

Deliberately not used: no map tile provider, no cloud services, no external APIs, no API keys. A clean checkout runs offline, because the target user is on a ship below 60°S where there is no geostationary coverage.

12. The models

Sea ice

Concentration comes from an ice-edge climatology with an austral seasonal cycle, multi-octave noise for floes and leads, and coastal polynyas that *emerge* where katabatic wind drives ice offshore rather than being painted in. The floe pattern is sampled in a drifting frame, so a lead is the same lead a day later and tens of kilometres downstream — which is what makes a Lagrangian forecast the physically correct method rather than a decorative one.

Thickness answers the question every reviewer asks. Altimeters (CryoSat-2, ICESat-2) have narrow swaths and long repeat cycles and cannot support tactical navigation. So thickness is derived thermodynamically from accumulated Freezing Degree Days by Stefan's Law, saturating near 1.55 m where oceanic heat flux balances conduction through the ice and its snow, plus a dynamic ridging term where the drift field converges. Pure Stefan growth diverges as √FDD and would wrongly predict 3 to 4 m by late season.

Divergence is the besetting predictor. Where the drift field converges, floes are driven together, leads close behind the ship and pressure builds against the hull. The route cost penalises it and the voyage engine alerts on it.

Forecast verification — the number that matters most

Any forecast must beat persistence — "assume nothing changes" — or it is adding nothing. The forecast never sees the verifying analysis.

Lead	Forecast RMSE	Persistence RMSE	Skill	IIEE	Persistence IIEE
24 h	0.0811	0.0877	+0.075	0.046	0.041
48 h	0.1094	0.1279	+0.145	0.053	0.057
72 h	0.1258	0.1502	+0.163	0.050	0.067
120 h	0.1373	0.1659	+0.172	0.050	0.070
168 h	0.1519	0.1817	+0.164	0.058	0.078

Concentration units. Positive skill means the forecast beat persistence. Simulated-environment skill.

Ship performance — Lindqvist (1989)

Crushing, bending and submergence resistance with the correct velocity corrections, plus open-water resistance via the ITTC-1957 line so total resistance stays continuous as thickness approaches zero. Four errors in the original transcription of the method were found and fixed, including a bending term that was dimensionally wrong and *backwards in Young's modulus*, making stiffer ice easier to break.

Speed is an output, never an assumption. Every route segment solves the lower of the power balance and the propeller thrust balance. The thrust limit binds in heavy ice, and ignoring it silently assumes a propeller that keeps open-water efficiency down to bollard conditions, which no propeller does.

Iceberg drift

Fourth-order Runge-Kutta on the momentum balance `(M + M_a) dv/dt = F_air + F_water + F_coriolis + F_pressure + F_wave`, with deterioration by basal turbulent melt, buoyant convection and wave erosion, and perturbed ensembles giving 50 and 90 percent positional uncertainty per lead time. The timestep is derived per berg from its own drag response timescale.

Near-field radar

A simulated X-band plan-position indicator whose honesty is the point: it reports what it *missed*. In a representative pack-ice sweep there are 39 real targets, 13 are painted and 26 go undetected, one of them inside the three-mile alert

perimeter. Growler detection range collapses from 3.0 nm in calm water to 0.6 nm in saturated clutter.

13. Machine learning: what was trained, and what failed

Trained full run, 252 s wall time, scikit-learn 1.3.2. Every figure is in `models/metrics.json`.

Model	Task	Result	Used?
Growler / clutter classifier	Real ice target vs sea-clutter false alarm	F1 **0.989**, ROC-AUC 0.999, against 0.701 for an SNR threshold	Yes
Sea-ice concentration forecaster	Concentration at +24 to +168 h	Test RMSE 0.105; skill vs persistence **-0.006**, vs physics **-0.072**, vs climatology **+0.285**	No
Iceberg drift residual corrector	Velocity error of the physics model	72 h position error 6.95 km → 6.86 km; mean +1.3%, median **-12.9%**	No

> **Two of the three trained models do not beat the physics they were meant to improve, and are not used.** This is reported rather than hidden, because a system claiming three working AI engines when one works is a system nobody should trust with a ship.
>
> **Why they fail, honestly.** The training data is generated by this repository's own physics environment. The synthetic ice field's patterns are *produced* by advection, so a semi-Lagrangian back-trajectory is close to the exact inverse of the generating process and no gradient-boosted tree on tabular features will beat it in its own world. The iceberg "observations" are generated by perturbing the same momentum balance being corrected, leaving the residual close to unstructured noise. On real Copernicus rasters and real USNIC tracks — where the true dynamics are far richer and the physics has genuine systematic bias — these are exactly the places a learned model earns its keep, which is why the pipeline is built and kept.
>
> The classifier works because separating a small ice target from a clutter spike is genuinely multivariate: aspect ratio alone carries 46% of the feature importance, and no single threshold can use it.

Splits are time-based with an embargo, never random. Nearby samples share the same underlying noise field, so a random split would leak the answer and report a fictitious score.

14. Constraints the system is built around

Constraint	How the build answers it
No geostationary coverage below 60°S	Contours and deltas instead of rasters. **Measured** at 23.4 KB/day against a 50 KB Iridium Certus budget — both payloads are built, gzipped and the byte counts read, not estimated.
Polar night and cloud for months	Reliance on C-band SAR and passive microwave, unaffected by darkness or cloud. No optical product is in the critical path.
Regulation forbids unapproved software in a certified ECDIS	The console runs on separate hardware and exports GPX 1.1, GeoJSON and an S-411-shaped ice overlay over the ship's LAN, loaded read-only. The S-411 output is explicitly a representative subset, not a certified encoding.
No GPU on a ship's bridge	Every model is CPU-only. The heaviest training run completes in about four minutes on a laptop.
A lead can close in under an hour	Drift-field divergence gives a compression index; convergent regimes are penalised in the route cost and raise an alert under way.
Growlers are invisible to satellites	A separate near-field radar layer that reports its own misses rather than implying complete detection.
The forecast will be wrong on arrival	The voyage engine resamples conditions at the ship's actual arrival time and re-plans from the present position when a hard constraint is violated.
Ice thickness cannot be measured tactically	Thermodynamic derivation from Freezing Degree Days plus dynamic ridging, as operational ice services do.
A demonstration must not stall	Caches are warmed at start-up in a worker thread. The first plan a user waits for dropped from 29 s to 11.7 s, and the server answers immediately.

15. Engineering: performance, and the bugs it exposed

Problem	Fix	Result
Land test in the A* inner loop	Bounding-box reject plus a memo cache on the quantised lattice	25,000 queries/s
Coast distance on gridded fields	One batched KD-tree query into a bilinear lookup grid	200k samples in 67 ms
Ice model re-requesting static coast geometry	Cache keyed on grid content, plus 48-hour forecast slices	planning 24 s → 15 s
Attainable speed per A* edge	Tabulate over thickness and concentration, then interpolate	about 1000×
Iceberg ensemble integration	Vectorise the ensemble; refresh forcing every 30 simulated minutes	109 s → 4 s
Voyage tick	Reuse the sampled state in the radar sweep; refresh the berg check on an interval	2340 ms → 169 ms
Cold start before the first plan	Warm caches and pre-plan the default leg at start-up, in a worker thread	29 s → 11.7 s

Three defects worth recording, because they were correctness bugs, not slow code

1. **The iceberg integrator diverged to NaN for small bergs.** A fixed timestep is stable for a giant tabular berg whose drag response time is many hours, but a growler responds in minutes; the velocity oscillated and blew up within a simulated day. The timestep is now derived per berg from its own response timescale.
2. **The planner certified routes that beset the ship.** The route optimiser inverted the power curve directly while the voyage engine used the full thrust-limited solver, so the planner was systematically optimistic. Two components disagreeing about the same physics is worse than either being wrong alone. Both now call one function.
3. **Iceberg avoidance tested the wrong position.** Keep-out zones used each berg's position at departure, but a tabular berg drifts tens of kilometres a day. On a 300-hour passage a route planned clear of D-30A still passed 3.0 nm from the *centre* of a 32 km berg — that is inside it. Exclusion zones now follow each berg's forecast track, interpolated to the ship's arrival time. The closest approach went from 3.0 nm to 160 nm.

16. Honest limitations

1. **Environmental fields are synthetic.** All skill figures are simulated-environment figures and must be described that way.
2. **Two of three trained models lose to their physics baselines** and are not in the serving path.
3. **Route optimality is approximate in time.** Edge costs depend on arrival time; each node carries the arrival time of the best path found so far. This is standard in operational weather routing but is a label-correcting approximation, not a proof of optimality.
4. **Forecasts clamp at 240 hours.** Longer passages fall back to climatology in their later legs — correct, since nobody can forecast ice thirty days out, but it means very slow baseline routes are evaluated against climatology late on.
5. **The S-411 export is not certified.** It is a representative attribute subset; conformance requires an S-100 exchange set and validation against the IHO feature catalogue.
6. **Hull angles, block coefficients and fuel consumption are engineering estimates**, marked as such in the source. Principal dimensions and installed power are published figures.
7. **The 15–22% fuel reduction in the original blueprint is not reproduced.** That figure came from the literature, not from this system.

8. **No authentication or multi-tenancy.** It is a single-console prototype.

17. How to run and verify it

```
# One command
.\start.ps1                    (Windows)      ./start.sh          (macOS, Linux)

# Or by hand
pip install -r requirements.txt
uvicorn src.api.main:app --port 8000          → http://localhost:8000/docs
cd frontend && npm install && npm run dev      → http://localhost:5173

# Verify
python -m pytest tests/ -q                    → 108 passed
python -m src.cli                             → the whole system in a terminal
python -m scripts.train                       → retrain every model
```

The README contains a six-step verification walkthrough giving the expected output of each check, so a reviewer can tell the difference between “it started” and “it works”.

18. Standards and references

Standards and law: IMO MSC.385(94) Polar Code · IMO MSC.1/Circ.1519 POLARIS · IHO S-411 sea-ice product specification · Indian Antarctic Act, 2022.

Data products the production system would ingest: EUMETSAT OSI SAF OSI-401-b · AMSR2 (JAXA GCOM-W1) / University of Bremen ASI · ESA Sentinel-1 GRD EW · ECMWF ERA5 and HRES · Copernicus Marine GLOBAL_ANALYSISFORECAST_PHY_001_024 · US National Ice Center Antarctic iceberg database.

Literature: Lindqvist (1989), POAC · Bigg et al. (1997), *Cold Regions Science and Technology* · Rackow et al. (2017), *JGR Oceans* · El-Tahan et al. (1987) · Timco & Weeks (2010) · Andersson et al. (2021), *Nature Communications* (IceNet) · Anderson (1961), *Journal of Glaciology*.

POLAR-NAV AI v1.0.0 · generated 07 September 2026, 11:13 UTC · Python 3.11.9 · coastline: 97 polygons, 1698 vertices.
Every figure in this report was computed when the report was generated. Regenerate with `python -m scripts.make_report`.